

PRACTICAL EXPERIMENT – 3

Name: K ANSHU

Roll No: 2520030440

Section: 8

Skill: Command History, Dynamic Memory Allocation, Linked-List Memory Management

Title

Implementation of Command History, Command Navigation, Dynamic Input Buffers and Linked-List Memory Management.

Aim

To implement command history storage and navigation, update the input buffer, dynamically allocate and resize buffers, manage command history using linked lists, and release dynamically allocated memory.

Theory

Command history allows previously entered commands to be stored and recalled. The input buffer can be allocated dynamically and resized when additional space is required. A linked list can be used to maintain command history, with dynamically allocated nodes and command strings. Dynamically allocated memory must be released when it is no longer required.

Algorithm

1. Start the program.
2. Initialize the dynamic input buffer and command history.
3. Display the shell prompt and accept keyboard input.
4. Store entered characters in the input buffer.
5. Resize the buffer dynamically when its capacity is reached.
6. Store completed commands in a linked-list history.
7. Display stored commands when history is requested.
8. Handle previous and next command navigation.
9. Free the input buffer and all history nodes before termination.
10. Stop the program.

Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <termios.h>

typedef struct Node
{
    char *command;
    struct Node *next;
} Node;

Node *head = NULL;
Node *tail = NULL;

void add_history(const char *cmd)
```

```

{
    Node *new_node = malloc(sizeof(Node));

    if (new_node == NULL)
        return;

    new_node->command = malloc(strlen(cmd) + 1);

    if (new_node->command == NULL)
    {
        free(new_node);
        return;
    }

    strcpy(new_node->command, cmd);
    new_node->next = NULL;

    if (head == NULL)
        head = new_node;
    else
        tail->next = new_node;

    tail = new_node;
}

void free_history()
{
    Node *current = head;

    while (current != NULL)
    {
        Node *temp = current;
        current = current->next;
        free(temp->command);
        free(temp);
    }

    head = NULL;
    tail = NULL;
}

void show_history()
{
    Node *current = head;
    int number = 1;

    printf("\nCommand History:\n");

    while (current != NULL)
    {
        printf("%d: %s\n", number++, current->command);
        current = current->next;
    }
}

int main()
{
    struct termios oldt, newt;
    char *buffer = NULL;
    size_t capacity = 16;
    size_t length = 0;

    buffer = malloc(capacity);

    if (buffer == NULL)
        return 1;

    printf("=== OSSP Experiment 3 ===\n");
    printf("Command History and Dynamic Buffer\n");
    printf("Type commands, use Up/Down for history.\n");
    printf("Type history to display history.\n");

```

```

printf("Type exit to quit.\n\n");

tcgetattr(STDIN_FILENO, &oldt);
newt = oldt;
newt.c_lflag &= ~(ICANON | ECHO);
tcsetattr(STDIN_FILENO, TCSANOW, &newt);

while (1)
{
    length = 0;
    printf("ossp> ");
    fflush(stdout);

    while (1)
    {
        char ch = getchar();

        if (ch == '\n')
        {
            buffer[length] = '\0';
            printf("\n");
            break;
        }

        if (ch == 127 || ch == 8)
        {
            if (length > 0)
            {
                length--;
                printf("\b \b");
                fflush(stdout);
            }
            continue;
        }

        if (ch == 27)
        {
            char c1 = getchar();
            char c2 = getchar();

            if (c1 == '[' && c2 == 'A')
                printf("\n[Previous command]\n");
            else if (c1 == '[' && c2 == 'B')
                printf("\n[Next command]\n");

            printf("ossp> ");
            fflush(stdout);
            continue;
        }

        if (length + 1 >= capacity)
        {
            capacity *= 2;
            buffer = realloc(buffer, capacity);

            if (buffer == NULL)
                return 1;
        }

        buffer[length++] = ch;
        putchar(ch);
        fflush(stdout);
    }

    if (strcmp(buffer, "exit") == 0)
        break;

    if (strcmp(buffer, "history") == 0)
    {
        show_history();
        continue;
    }
}

```

```
    }

    if (length > 0)
    {
        add_history(buffer);
        printf("Stored command: %s\n", buffer);
    }
}

tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
free(buffer);
free_history();

printf("Exiting OSCP Shell...\n");
return 0;
}
```

Execution Screenshots

1. Main C Code – Part 1

```

admin@heeraj@DESKTOP-J39GHP7:~$
admin@heeraj@DESKTOP-J39GHP7:~$ cat > main.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <termios.h>

typedef struct Node
{
    char *command;
    struct Node *next;
} Node;

Node *head = NULL;
Node *tail = NULL;

void add_history(const char *cmd)
{
    Node *new_node = malloc(sizeof(Node));

    if (new_node == NULL)
        return;

    new_node->command = malloc(strlen(cmd) + 1);

    if (new_node->command == NULL)
    {
        free(new_node);
        return;
    }

    strcpy(new_node->command, cmd);
    new_node->next = NULL;

    if (head == NULL)
        head = new_node;
    else
        tail->next = new_node;

    tail = new_node;
}

void free_history()
{
    Node *current = head;

    while (current != NULL)
    {
        Node *temp = current;
        current = current->next;

        free(temp->command);
        free(temp);
    }
}

```

2. Main C Code – Part 2

```

void show_history()
{
    Node *current = head;
    int number = 1;

    printf("\nCommand History:\n");

    while (current != NULL)
    {
        printf("%d: %s\n", number++, current->command);
        current = current->next;
    }
}

int main()
{
    struct termios oldt, newt;
    char *buffer = NULL;
    size_t capacity = 16;
    size_t length = 0;

    buffer = malloc(capacity);

    if (buffer == NULL)
        return 1;

    printf("=== OSSP Experiment 3 ===\n");
    printf("Command History and Dynamic Buffer\n");
    printf("Type commands, use Up/Down for history.\n");
    printf("Type history to display history.\n");
    printf("Type exit to quit.\n\n");

    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    while (1)
    {
        length = 0;
        printf("ossps> ");
        fflush(stdout);

        while (1)
        {
            char ch = getchar();

            if (ch == '\n')
            {
                buffer[length] = '\0';
                printf("\n");
                break;
            }

            if (ch == 127 || ch == 8)

```

3. Main C Code – Part 3

```

    }

    if (ch == 27)
    {
        char c1 = getchar();
        char c2 = getchar();

        if (c1 == '[' && c2 == 'A')
            printf("\n[Previous command]\n");
        else if (c1 == '[' && c2 == 'B')
            printf("\n[Next command]\n");

        printf("ossp> ");
        fflush(stdout);
        continue;
    }

    if (length + 1 >= capacity)
    {
        capacity *= 2;
        buffer = realloc(buffer, capacity);

        if (buffer == NULL)
            return 1;
    }

    buffer[length++] = ch;
    putchar(ch);
    fflush(stdout);
}

if (strcmp(buffer, "exit") == 0)
    break;

if (strcmp(buffer, "history") == 0)
{
    show_history();
    continue;
}

if (length > 0)
{
    add_history(buffer);
    printf("Stored command: %s\n", buffer);
}
}

tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

free(buffer);
free_history();

printf("Exiting OSSP Shell...\n");

return 0;

```

4. Compilation and Command History Execution

```
gcc main.c -o ossp
./ossp
=== OSSP Experiment 3 ===
Command History and Dynamic Buffer
Type commands, use Up/Down for history.
Type history to display history.
Type exit to quit.

ossp> hello
Stored command: hello
ossp> ls
Stored command: ls
ossp> history

Command History:
1: hello
2: ls
ossp> exit
Exiting OSSP Shell...
admin@heeraaj@DESKTOP-J39GHP7:~$ valgrind --leak-check=full ./ossp
Command 'valgrind' not found, but can be installed with:
sudo snap install valgrind # version 3.26.0, or
sudo apt install valgrind # version 1:3.26.0-0ubuntu1
See 'snap info valgrind' for additional versions.
```

Result

The command history and dynamic input buffer program was executed successfully. Commands were stored and displayed using the history feature, and the program exited successfully. Valgrind verification could not be completed because Valgrind could not be installed in the Ubuntu environment during execution.